

04- Load Balancing

What is load balancing?

Load balancing is the method of distributing network traffic equally across a pool of resources that support an application. Modern applications must process millions of users simultaneously and return the correct text, videos, images, and other data to each user in a fast and reliable manner. To handle such high volumes of traffic, most applications have many resource servers with duplicate data between them. A load balancer is a device that sits between the user and the server group and acts as an invisible facilitator, ensuring that all resource servers are used equally.

What are the benefits of load balancing?

Application availability

Problem to solve: Server failure or maintenance can increase application downtime, making your application unavailable to visitors.

Load balancers increase the fault tolerance of your systems by automatically detecting server problems and **redirecting client traffic to available servers**. You can use load balancing to make these tasks easier:

- Run application server maintenance or upgrades without application downtime
- Provide automatic disaster recovery to backup sites
- Perform health checks and prevent issues that can cause downtime

Application scalability

You can use load balancers to direct network traffic intelligently among multiple servers. Your applications can handle **thousands of client requests** because load balancing does the following:

- Prevents traffic bottlenecks at any one server
- Predicts application traffic so that you can add or remove different servers, if needed
- Adds redundancy to your system so that you can scale with confidence

Application security

Load balancers come with built-in security features to add another layer of security to your internet applications. They are a useful tool to **deal with distributed denial of service attacks (DDoS)**, in which **attackers flood an application server with millions of concurrent requests that cause server failure**. Load balancers can also do the following:

- Monitor traffic and block malicious content
- Automatically redirect attack traffic to multiple backend servers to minimize impact
- Route traffic through a group of network firewalls for additional security

Application performance

Load balancers improve application performance by increasing response time and reducing network latency. They perform several critical tasks such as the following:

- Distribute the load evenly between servers to improve application performance
- Redirect client requests to a geographically closer server to reduce latency
- Ensure the reliability and performance of physical and virtual computing resources

What are the types of load balancing technology?

Load balancers are one of two types: hardware load balancer and software load balancer.

Hardware load balancers

A hardware-based load balancer is a hardware appliance that can securely process and redirect gigabytes of traffic to hundreds of different servers. You can store it in your data centers and use virtualization to create multiple digital or virtual load balancers that you can centrally manage.

Software load balancers

Software-based load balancers are applications that perform all load balancing functions. You can install them on any server or access them as a fully managed third-party service.

Comparison of hardware balancers to software load balancers

Hardware load balancers **require an initial investment, configuration, and ongoing maintenance**. You might also **not use them to full capacity**, especially if you purchase one only to handle peak-time traffic spikes. If traffic volume increases suddenly beyond its current capacity, this will affect users until you can purchase and set up another load balancer.

In contrast, **software-based load balancers are much more flexible**. They can scale up or down easily and are more compatible with modern cloud computing environments. They also **cost less** to set up, manage, and use over time.

Problem

- Past a certain point, web applications outgrow a single server deployment.
- Companies either want to increase their availability
- To do this, they deploy their application across multiple servers with a load balancer in front to distribute incoming requests.
- Big companies may need thousands of servers running their web application to handle the load.

Visualising the problem

Case1 : Let's start at the beginning: a single load balancer sending requests to a single server. Requests are being sent at a rate of **1 request per second (RPS)**, and each request reduces in size as the server processes it. For a lot of websites, this setup works just fine. Modern **servers** are powerful and can handle a lot of **requests**.

Case 2: But what happens when request are more than server capabilities?

Here we see that a rate of 3 RPS causes some **requests** to get **dropped**.

If a **request** arrives at the **server** while another **request** is being processed, the **server** will **drop** it (Shown as red circle).

This will result in an error being shown to the user.



Different Types of Load Balancing Algorithms

Load balancers can distribute requests to specific servers based on various load balancing techniques that use different algorithms to select servers based on a particular configuration. Load balancing algorithms can be divided into two categories:

- **Static load balancing algorithms:**

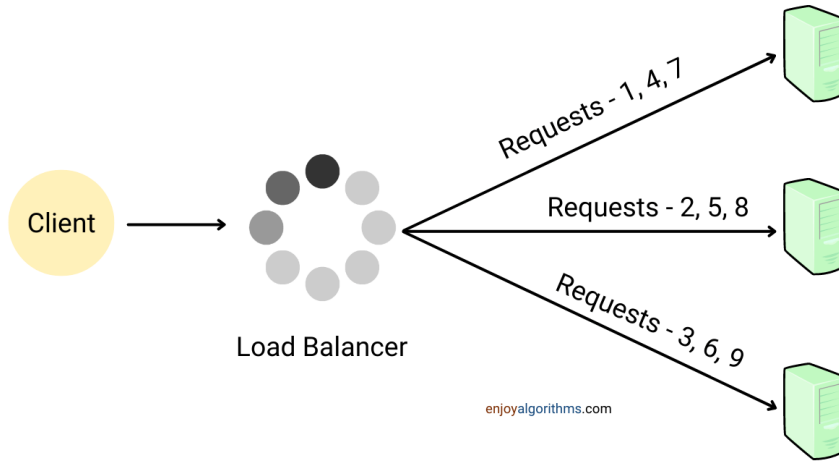
- a. These algorithms work the **same way regardless of the state of the backend server** serving the requests.
- b. They are **simpler** to implement, but they can lead to an **uneven distribution of requests**.
- c. Examples of static load balancing algorithms include round robin, weighted round robin, source IP hash, URL hash, randomized algorithm, etc.

- **Dynamic load balancing algorithms:**

- a. These algorithms take into account the **state of the backend server and consider server load** when distributing requests.
- b. They require communication between the load balancers and servers, so they are **slightly more complex but can distribute requests efficiently**.
- c. Examples of dynamic load balancing algorithms include the least connection method, weighted least connections method, least response time method, etc.

1- Round Robin Load Balancing Algorithm

Round robin is a simple load balancing algorithm that directs client requests to different servers based on a rotating list. The load balancer maintains a list of available servers and directs incoming requests in a round-robin fashion: the first request goes to the first server, the second request goes to the second server, and so on. When the load balancer reaches the end of the list, it goes back to the beginning and starts from the first server again.

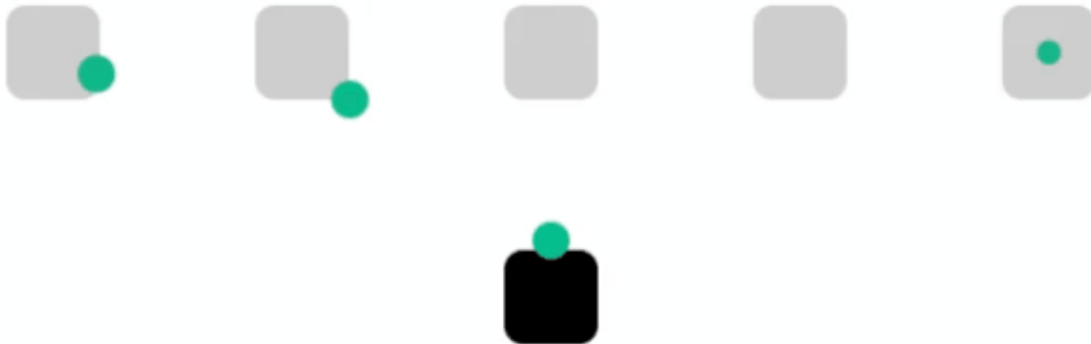


Solution

We can add another **server** to our **load balancer** to fix this.

Now, No more **dropped requests**!

The way our **load balancer** is behaving here, sending a request to each **server** in turn, is called "round robin" load balancing. It's one of the simplest forms of load balancing, and works well when your **servers** are all equally powerful and your **requests** are all equally expensive.



round robin algorithm problem

- In the real world, it's rare for **servers** to be equally powerful and **requests** to be equally expensive.
- Even if you use the exact same **server** hardware, performance may differ.
- Applications may have to service many different types of **requests**, and these will likely have different performance characteristics.
- Let's see what happens when we vary **request** cost. In the following simulation, **requests** aren't equally expensive. You'll be able to see this by some **requests** taking longer to shrink than others.



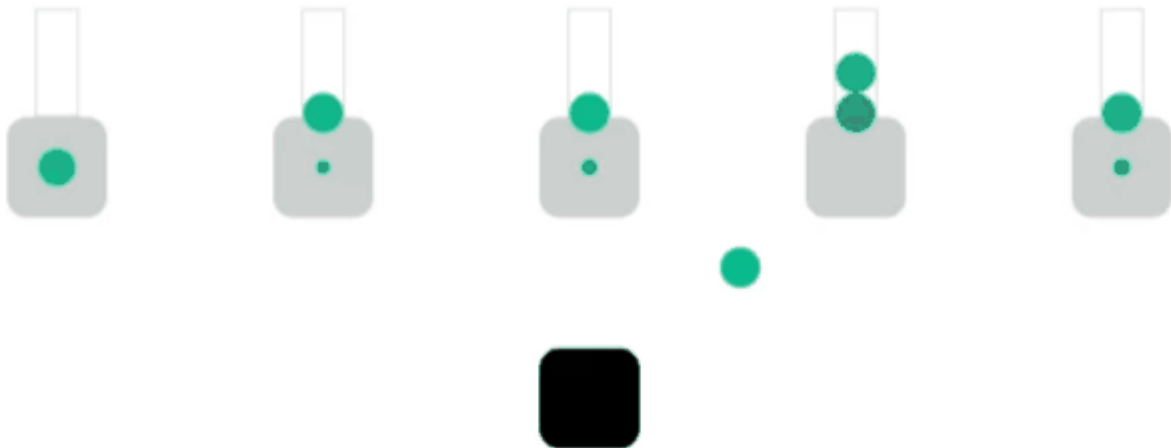
Benefits and drawbacks of the round robin algorithm:

1. It is easy to implement.
2. It evenly balances **traffic (not load)** between servers.
3. It does not consider the server's load or specifications, so there is a risk that a server with low capacity will receive a large number of requests and become overloaded.
4. It works best if every server in the load balancer list has roughly the same specification. Otherwise, a low processing server may have the same load as a high processing server.

Possible Solution: "request queue."

While most **requests** get served successfully, we do **drop** some. One of the ways we can mitigate this is to have a "request queue."

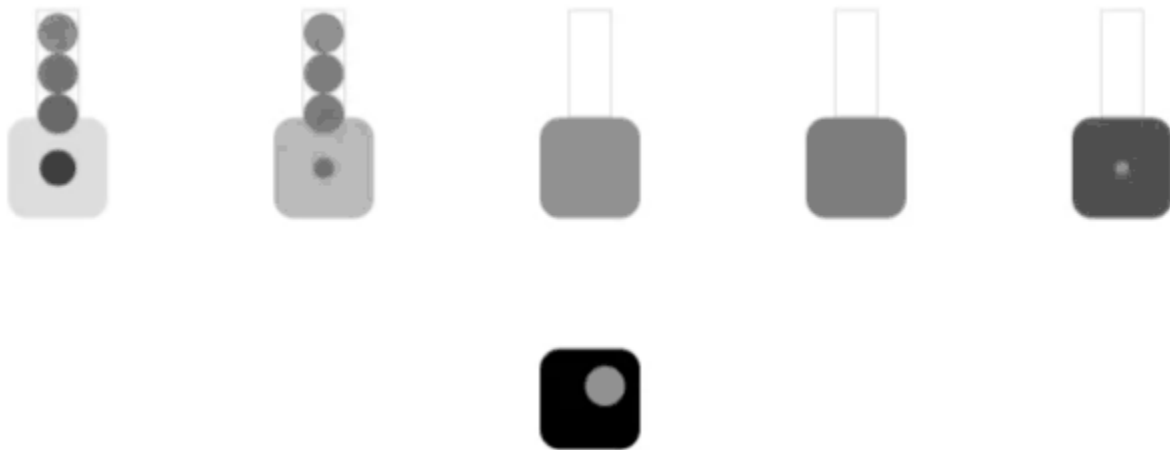
Request queues help us deal with uncertainty, but it's a trade-off. We will **drop** fewer **requests**, but at the cost of some **requests** having a **higher latency**.



Challenge with variable servers

Everything said above applies equally to **servers** that vary in power. In the next simulation we also vary the power of each **server**, which is represented visually with a **darker color = more powerful server**.

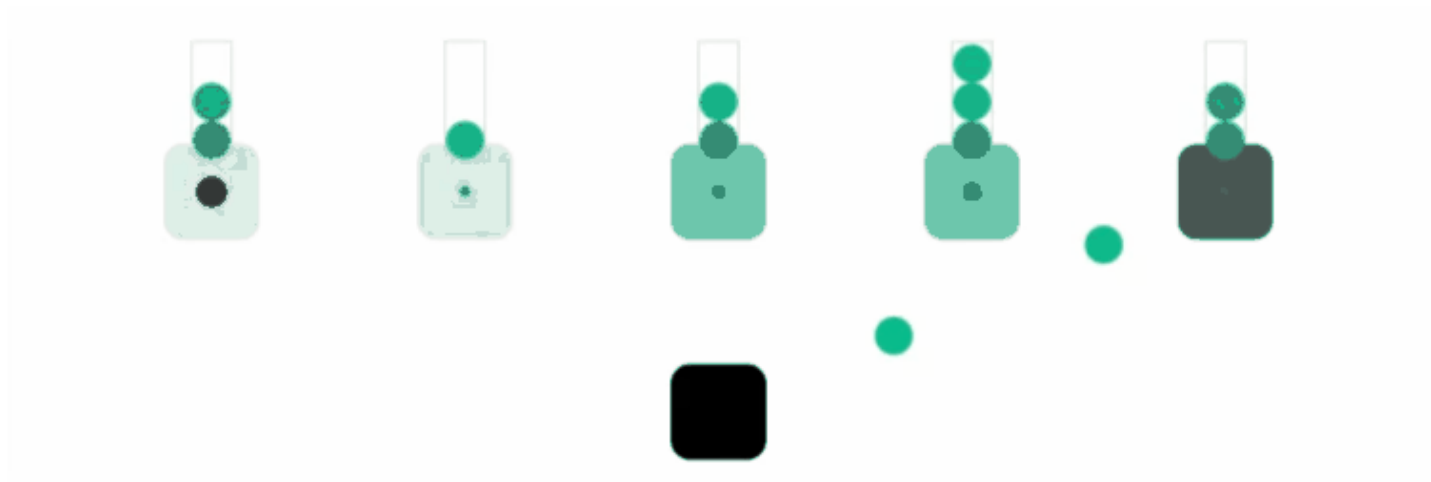
but odds are some are less powerful than others and quickly start to **drop requests**. At the same time, the more powerful **servers** sit idle most of the time. This scenario shows the key weakness of round robin: variance.



Improving on round robin

It's possible to tweak round robin to perform better with variance. There's an algorithm called "weighted round robin" which involves getting humans to tag each **server** with a weight that dictates how many **requests** to send to it.

In this simulation, we use each **server's** known power value as its **weight**, and we give more powerful **servers** more **requests** as we loop through them.



2- Weighted Round Robin Balancing Algorithm

The weighted round-robin load-balancing algorithm is a more advanced version of the simple round-robin algorithm. It distributes incoming requests based on the weighted score of the servers. The weight can be an integer that reflects the server's processing power or specifications. This allows the algorithm to consider server specifications when distributing traffic.

Some benefits and drawbacks of the weighted round-robin algorithm include:

1. It is slightly more complex than the simple round-robin algorithm, but it works well with servers with different specifications.
2. It does not consider the current load of each server or the relative computation cost of each request.
3. Based on the weighted score, some servers may receive a higher proportion of the overall request count.

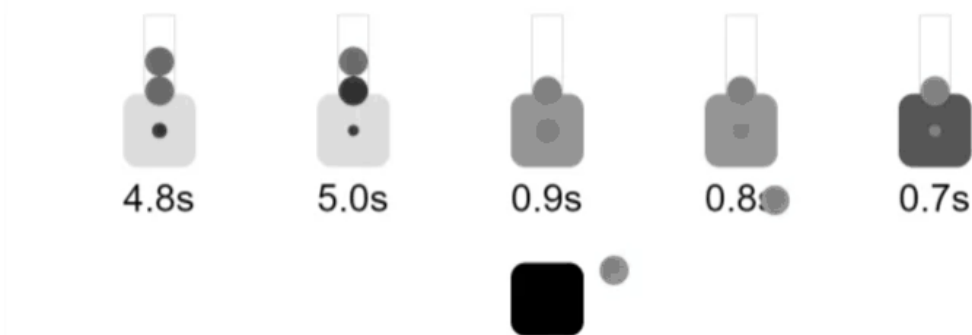
Problem of weighted round robin

While this handles the variance of **server** power better than vanilla round robin, we still have **request** variance to contend with.

In practice, getting humans to set the weight by hand falls apart quickly. Boiling **server** performance down to a single number is hard, and would require careful load testing with real workloads. This is rarely done, so another variant of weighted round robin calculates weights dynamically by using a **proxy metric: latency**.

It stands to reason that if one **server** serves **requests** 3 times faster than another **server**, it's probably 3 times faster and should receive 3 times more **requests** than the other **server**.

In the next figure: a text is added to each **server** this time that shows the average latency of the last 3 **requests** served.



3- Random Load Balancing Algorithm

The randomized load-balancing algorithm **randomly distributes requests** to servers using a **random number generator**.

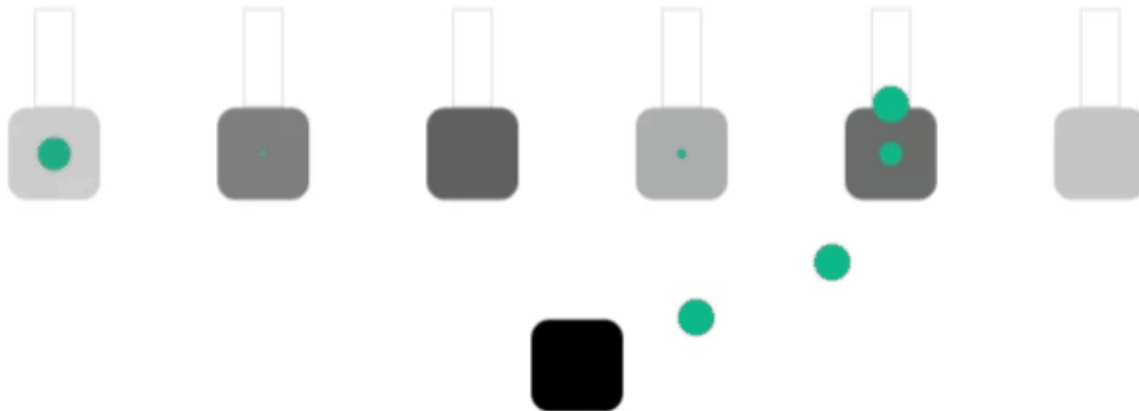
When a load balancer receives a request, the randomized algorithm evenly distributes it to the servers. Like the round robin algorithm, this algorithm works well for a group of servers with similar configurations.

4- "least connections" load balancing

Because the **load balancer** sits between the **server** and the user, it can accurately keep track of how many outstanding **requests** each **server** has. Then when a new **request** comes in and it's time to determine where to send it, it knows which **servers** have the least work to do and prioritises those.

This algorithm performs extremely well regardless how much variance exists. It cuts through uncertainty by maintaining an accurate understanding of what each **server** is doing. It also has the benefit of being very simple to implement.

While this algorithm is a great balance between simplicity and performance, it's **not immune** to **dropping requests**. However, what you'll notice is that the only time this algorithm **drops requests** is when there is literally no more queue space available.



Least Connection Method

The least connection load-balancing algorithm considers the current load on a server and aims to improve performance. It does this by sending requests to the server with the least number of active connections.

Some benefits and drawbacks of the least connection algorithm include:

1. The load balancer does additional calculations to determine the server with the least number of connections.
2. Problem: The additional calculations required to determine the server with the least number of connections may increase the load on the load balancer itself.

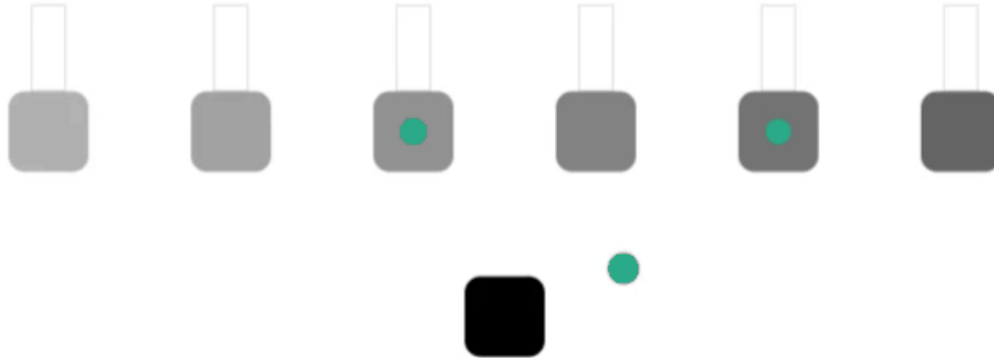
5-Weighted Least Connections Method

1. The weighted least connections load-balancing algorithm distributes load **based on both the number of current and active connections to each server and the relative capacity of the server.**
2. It takes into account that some servers can handle more connections than others.
3. Servers are rated based on their processing capabilities, allowing for a more fine-grained distribution of load.

6- PEWMA algorithm: "peak exponentially weighted moving average"

If you watch closely, you'll notice that the algorithm sends **fewer requests** to the leftmost **server** after a while. It does this because it figures out that all of the other **servers** are faster, and there's no need to send **requests** to the slowest one. That will just result in **requests** with a higher latency.

- For each **server**, the algorithm keeps track of the latency from the last N **requests**.
- Instead of using this to calculate an average, it sums the values but with an **exponentially** decreasing **scale** factor.
- This results in a value where the older a latency is, the less it contributes to the sum.
- Recent **requests** influence the calculation more than old ones.
- That value is then taken and multiplied by the number of open connections to the **server** and the result is the value we use to choose which **server** to send the next **request** to. Lower is better.



7- Source IP Hash Load Balancing Algorithm

The source IP hash load-balancing algorithm selects a server based on a unique hash key.

It generates the **hash** key **بناءً على معادلة** by **combining** the **source** and **destination** IP addresses and allocates the request to a specific server.

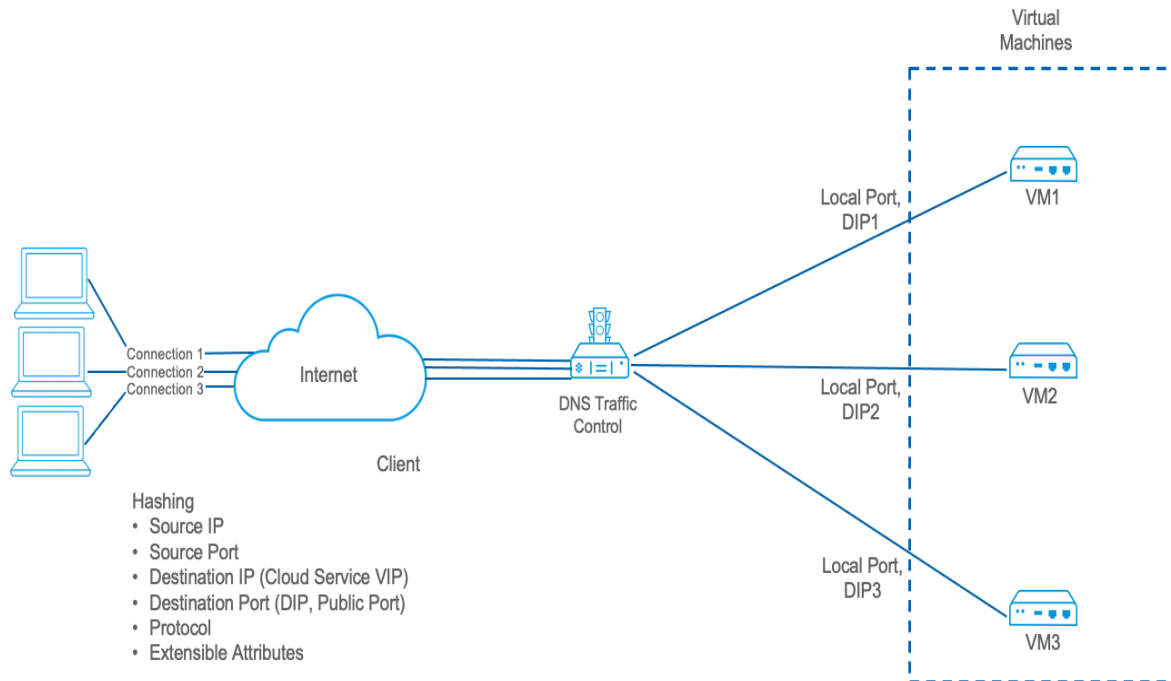
If the session is broken, the key can be regenerated and the client request can be directed back to the same server that was originally handling it.

In other words, this algorithm is useful when a dropped connection needs to be returned to the **same server**.

Advantages

Advantages

- Simple configuration
- persistence to the same application if connection is lost so the client can continue in the same active application.



8- URL Hash Load Balancing Algorithm

The URL hash load-balancing algorithm generates a **hash** value based on the **URL present in client requests**.

The requests are forwarded to servers based on the hash value.

The load balancer caches the hashed value of the URL, so subsequent requests that use the **same URL** result in a cache hit and are forwarded to the **same server**.

URL hash algorithm is useful when load-balanced servers serve mostly unique content per server. In other words, **all requests related to one process (e.g., running code) will go to one server**, and all requests related to another process (e.g., payments) will go to another server, and so on.

9- Min–Min algorithm

The basic Min-Min procedure in cloud environment selects the **task** with **least size** and chose a **Server** that has the **minimum capacity**.

The Min-Min algorithm is only suitable for small-scale Distributed systems

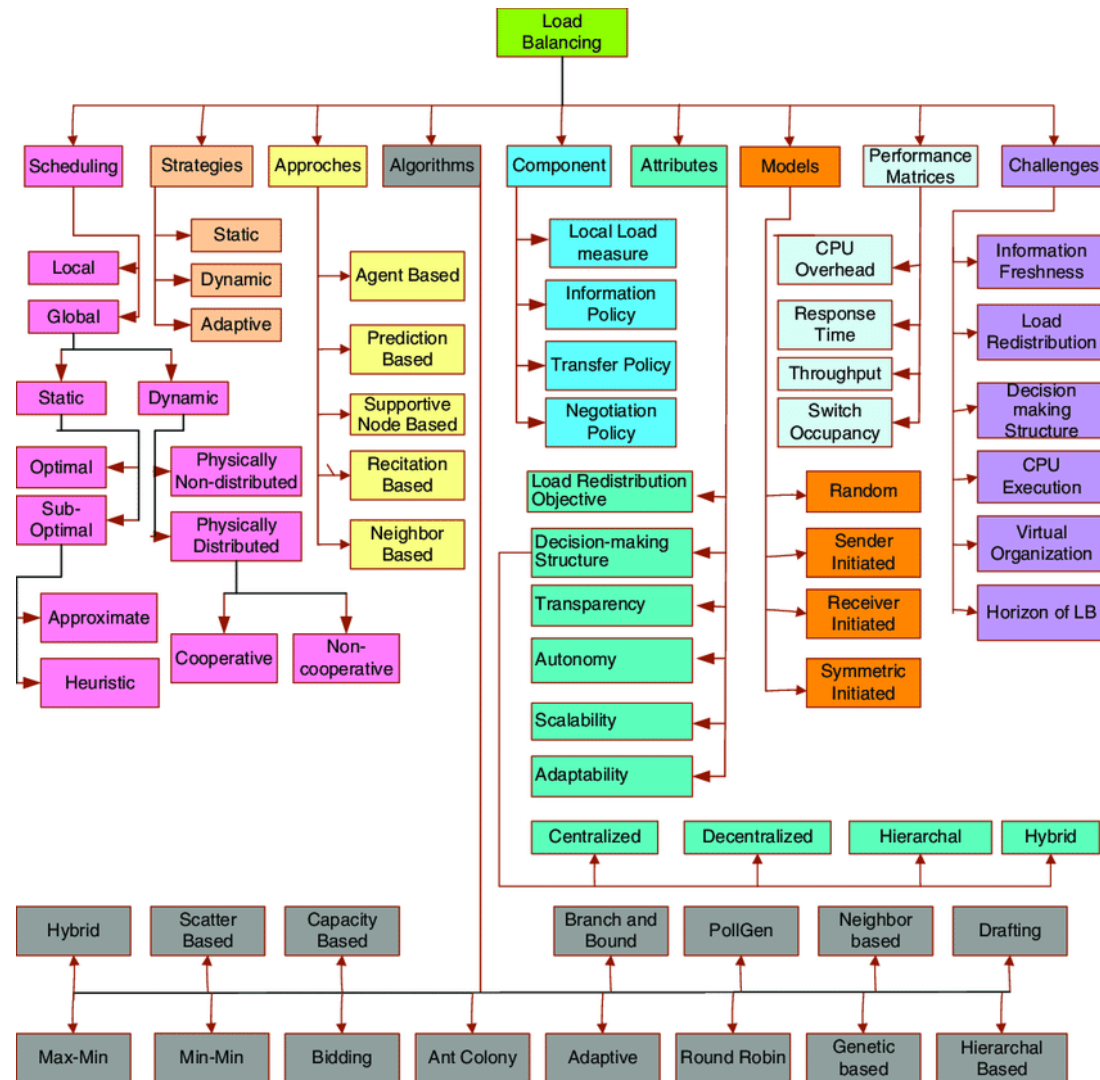
10- Genetic Algorithm (GA):

GA is based on the population and individual chromosome (Possible allocation) which may have some fitness values (like energy consumption, makespan, throughput, etc.) for optimizing them.

The basic GA algorithm performs Selection (e.g. Roulette wheel selection, Tournament selection, etc.), Crossover (e.g. Matrix Crossover, Single point crossover, etc.), Mutation (e.g. bit-wise mutation, boundary mutation, etc.) in each iteration. Most of the researchers considered the chromosome (is in vector form) size as the total number of tasks arrived at the system.

More Techniques

للاطلاع فقط



Sample Load Balancer in Java

Implementing the Load Balancer

We'll implement the load balancer using object-oriented programming principles. Our load balancer will utilize a minimum-load strategy to distribute requests evenly among a list of server instances.

Server Class

The `Server` class represents individual server instances and encapsulates essential attributes such as IP address and current load.

```
// Define a class for server instances
class Server {
    private String ipAddress;
    private int currentLoad;

    public Server(String ipAddress, int currentLoad) {
        this.ipAddress = ipAddress;
        this.currentLoad = currentLoad;
    }

    public String getIpAddress() {
        return ipAddress;
    }

    public int getCurrentLoad() {
        return currentLoad;
    }

    public void incrementLoad() {
        currentLoad++;
    }
}
```

Sample Load Balancer in Java-2

LoadBalancer Class

The **LoadBalancer** class manages multiple servers and distributes incoming requests among them.

- **addServer(String ipAddress, int load)**: Adds a new server to the load balancer with the specified IP address and load.
- **getNextServer()**: Implements a minimum load strategy to select the next server for routing requests. It increments the load and returns the IP address of the selected server.

```
public class LoadBalancer {
    private List<Server> servers;

    public LoadBalancer() {
        servers = new ArrayList<>();
    }

    // Method to add servers to the load balancer
    public void addServer(String ipAddress, int currentLoad) {
        servers.add(new Server(ipAddress, currentLoad));
    }

    // Method to get the next server IP address using minimum-load strategy
    public String getNextServer() {
        if (servers.isEmpty()) {
            throw new IllegalStateException("No servers available in the load balancer");
        }

        Server minLoadServer = servers.get(0);
        for (Server server : servers) {
            if (server.getCurrentLoad() < minLoadServer.getCurrentLoad()) {
                minLoadServer = server;
            }
        }

        // Increment the load of the selected server
        minLoadServer.incrementLoad();

        return minLoadServer.getIpAddress();
    }
}
```

Sample Load Balancer in Java-3

Main Class

Finally, we test our load balancer by adding some dummy server IP addresses and loads, and then retrieving the next server for routing requests using the Main Class.

Output

```
Request routed to server: 192.168.1.10  
Request routed to server: 192.168.1.11  
Request routed to server: 192.168.1.12
```

```
public static void main(String[] args) {  
    LoadBalancer loadBalancer = new LoadBalancer();  
  
    // Add some dummy server IP addresses and loads  
    loadBalancer.addServer("192.168.1.10", 10);  
    loadBalancer.addServer("192.168.1.11", 10);  
    loadBalancer.addServer("192.168.1.12", 10);  
  
    // Retrieve the next server for routing requests  
    String ipAddress1 = loadBalancer.getNextServer();  
    System.out.println("Request routed to server: " + ipAddress1);  
  
    String ipAddress2 = loadBalancer.getNextServer();  
    System.out.println("Request routed to server: " + ipAddress2);  
  
    String ipAddress3 = loadBalancer.getNextServer();  
    System.out.println("Request routed to server: " + ipAddress3);  
}
```

References

Load balancing algorithms

<https://samwho.dev/load-balancing/> (you can check the **interactive demo** at the **bottom** of that page)

<https://www.enjoyalgorithms.com/blog/types-of-load-balancing-algorithms>

<https://aws.amazon.com/what-is/load-balancing/>

<https://kemptechnologies.com/load-balancer/load-balancing-algorithms-techniques>